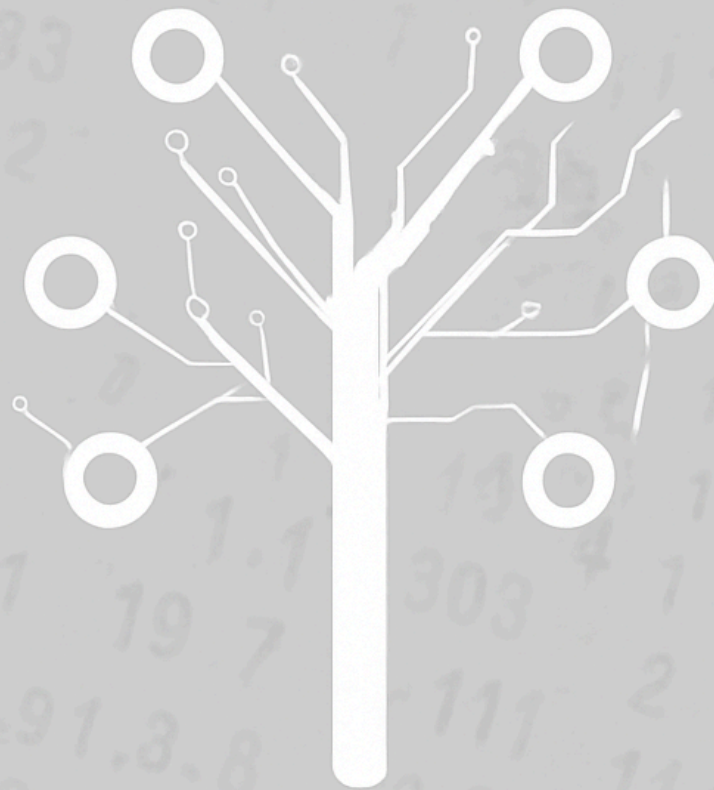


Presentazione tesi

Programmazione orientata agli oggetti nei modelli finanziari ad albero

Relatori: Sara Migliorni, Alessandro Gnoatto, Andrea Mazzon
Candidato: Carlo Andrea Tramentozzi
Laurea in Informatica L-31



Entità e interazioni

Prodotti finanziari

Alberi ricombinanti

Tecniche di
programmazione
dinamica e
implementazioni

Indice

1. Prodotti finanziari

1.1 Caratteristiche prodotti

1.2 Implementazione prodotti

1.3 Non Path Dependent

1.4 Path Dependent

2. Vantaggi dei modelli ad albero

3. Alberi

3.1 Alberi binomiali ricombinanti

3.2 Alberi trinomiali ricombinanti

4. Modelli utilizzati

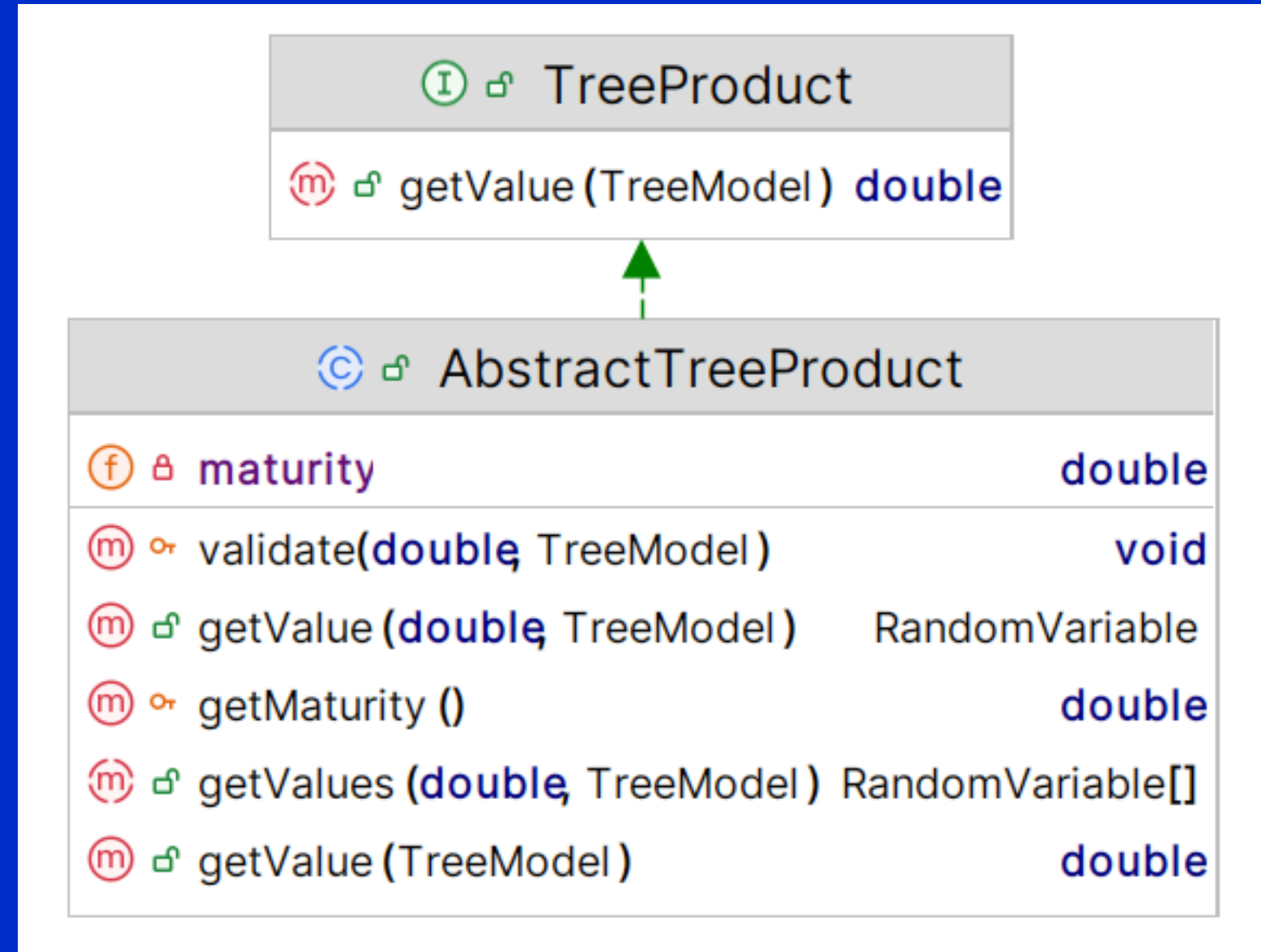
5. Gerarchia modelli

6. Tecniche di programmazione dinamica

7. Sfruttare la struttura dati

Prodotti finanziari

Le opzioni sono contratti che permettono di fare operazioni sul prezzo futuro di un mercato, dando il diritto, ma non l'obbligo di acquistare o vendere l'asset sottostante a un prezzo stabilito entro una determinata scadenza.



Caratteristiche prodotti

Non Path Dependent



Europee

Possibilità di esercizio
solo al tempo
terminale maturity

Americane

Facoltà di esercizio
anticipato in ogni nodo e
al tempo maturity, ad
ogni passo di ricorsione
si valuta il massimo tra
continuare ed esercitare
il diritto

Caratteristiche prodotti

Path Dependent



Asiatiche

Possibilità di esercizio a determinati fixing time e al tempo maturity.

Il valore dipende dalla media

Casi estremi:

Unico fixing time



Europea

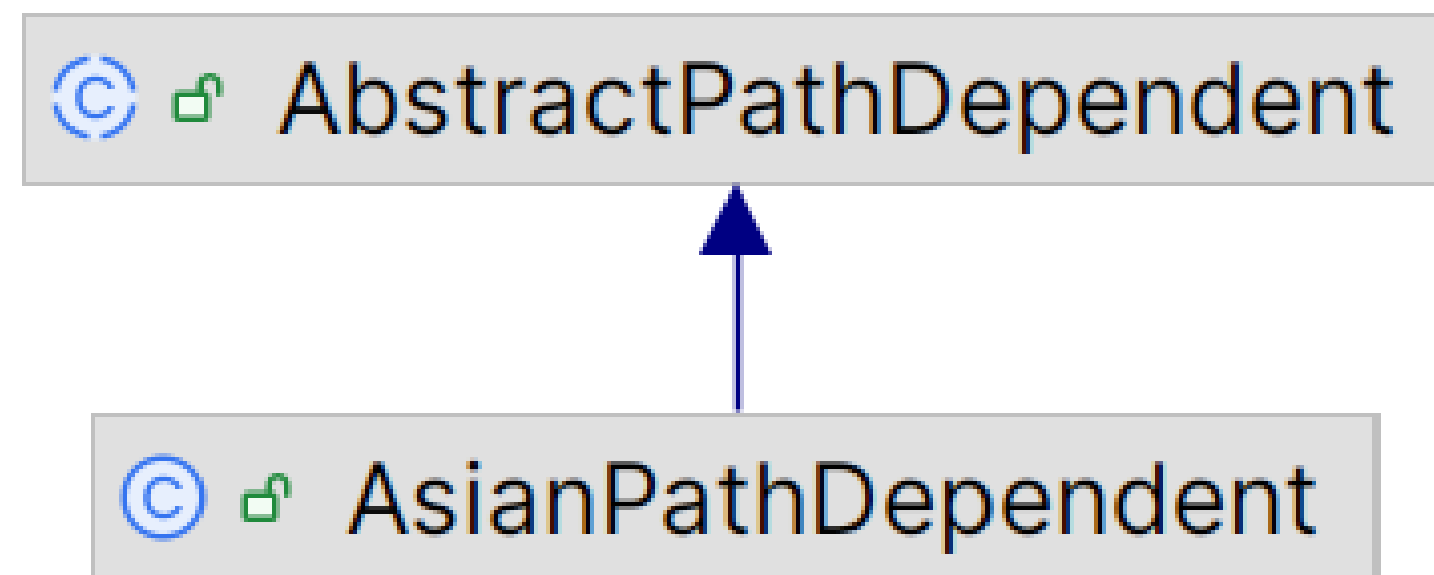
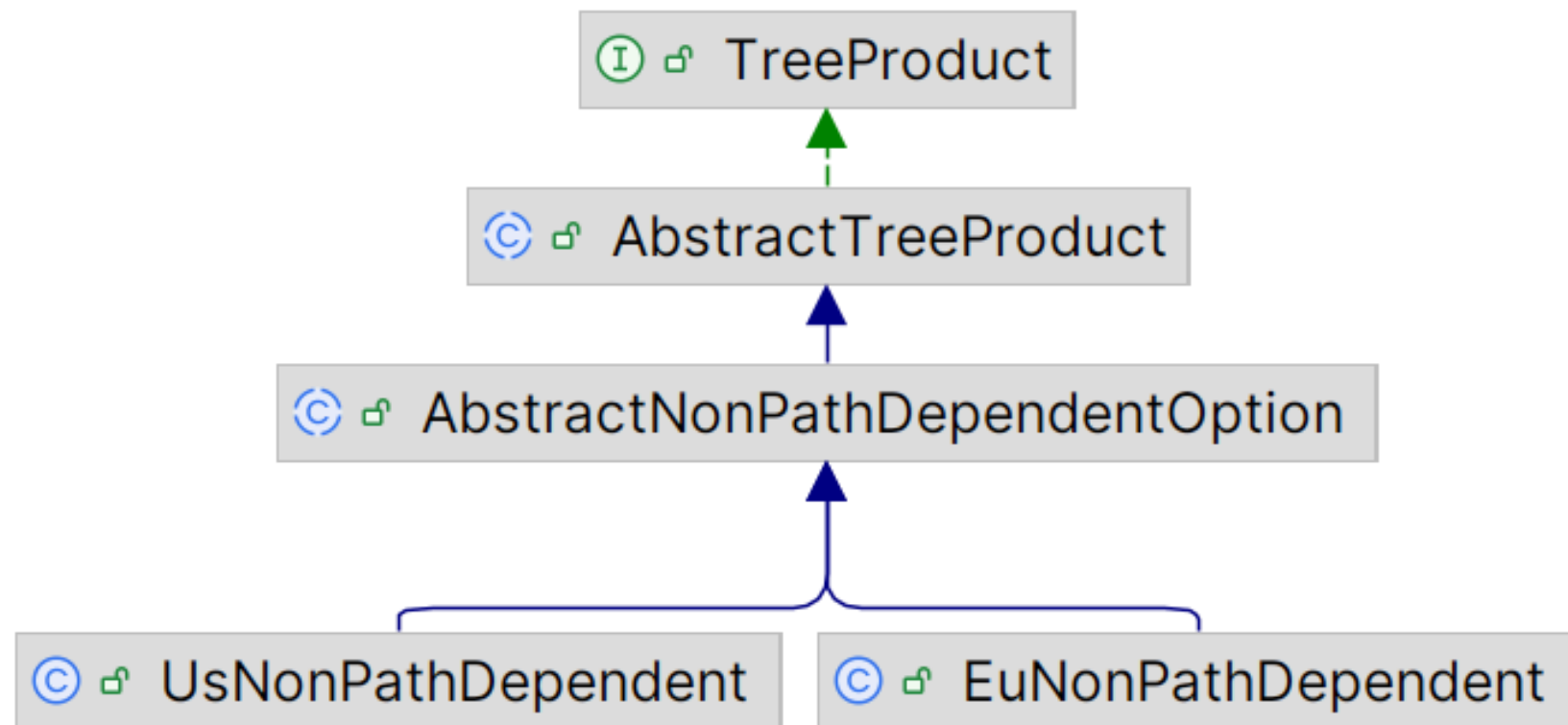
Fixing time ogni nodo



Americana

Implementazione prodotti

- Ogni prodotto fa riferimento all'interfaccia TreeProduct che dichiara il metodo getValue()
- Tutti i prodotti hanno una maturità, dunque fanno riferimento alla classe astratta AbstractTreeProduct
- I prodotti si specializzano in PathDependent come le asiatiche oppure in NonPathDependent come le europee e americane

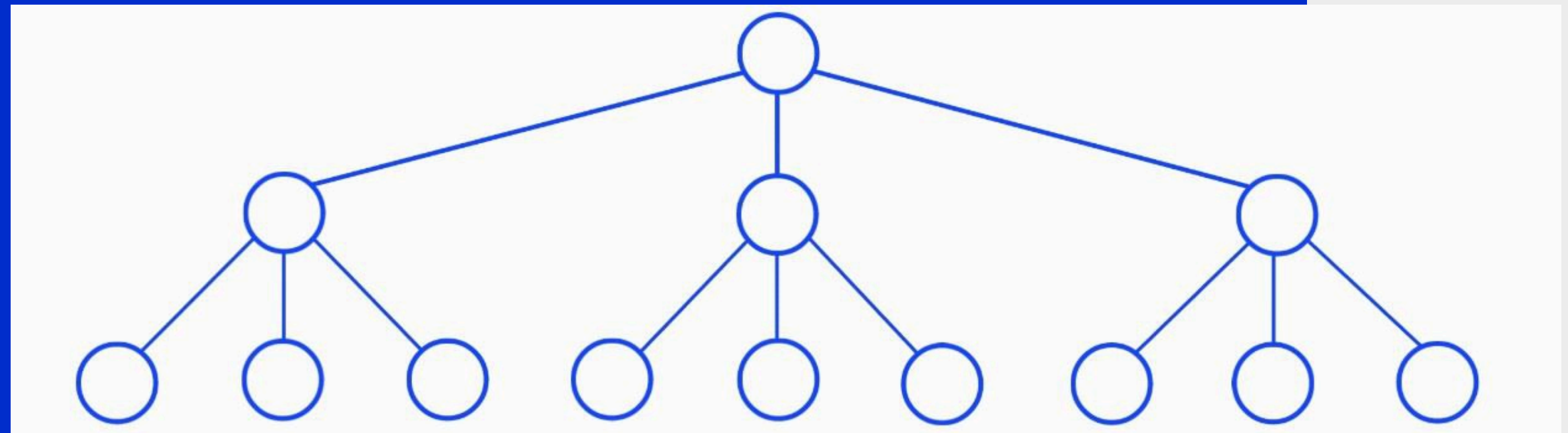
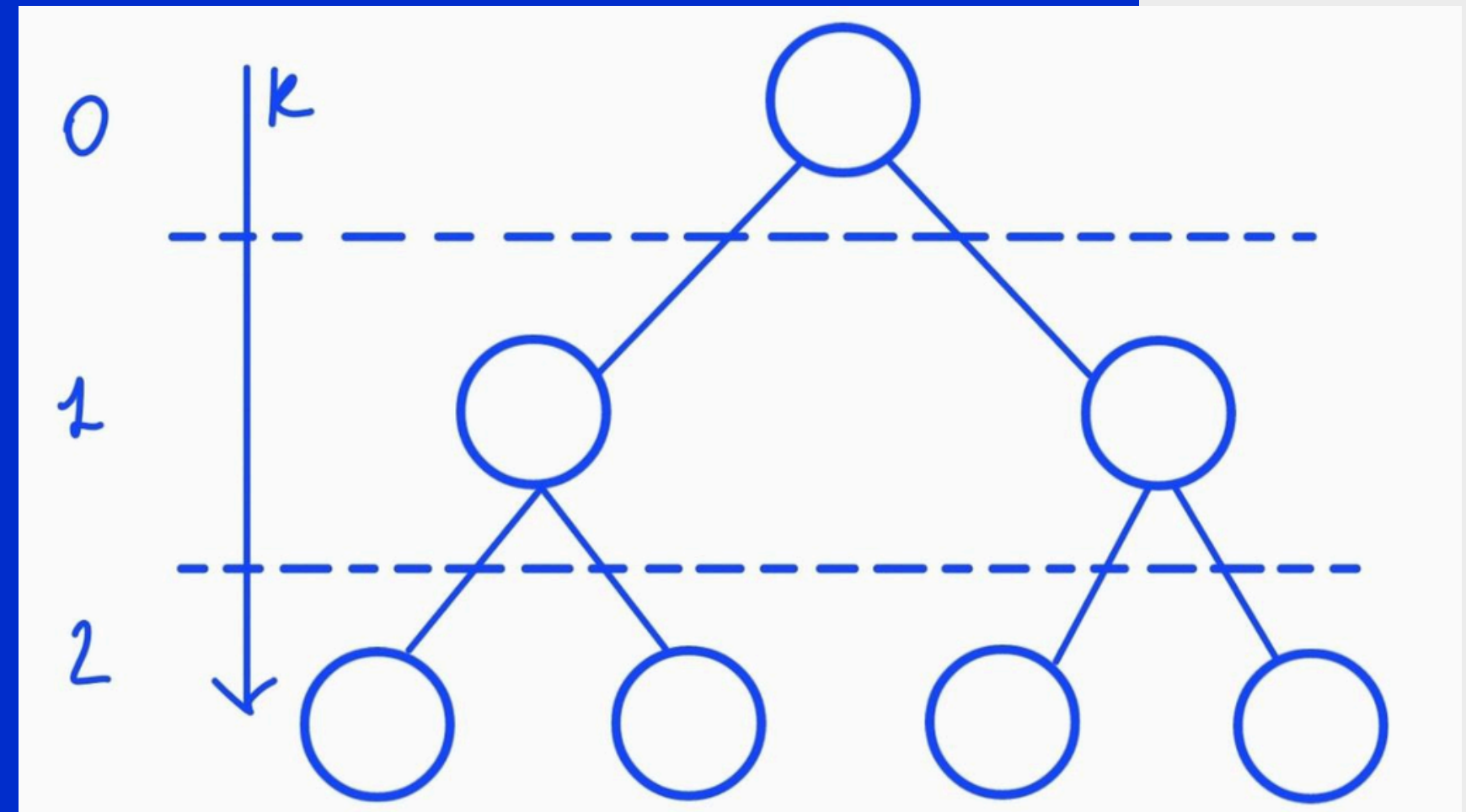


Vantaggi dei modelli ad albero

- Chiarezza
- Trasparenza
- Efficienza con albero ricombinante
- Facile implementazione dell'esercizio anticipato
- Convergenza a modelli continui aumentando gli step
- Implementazione leggera ed estendibile

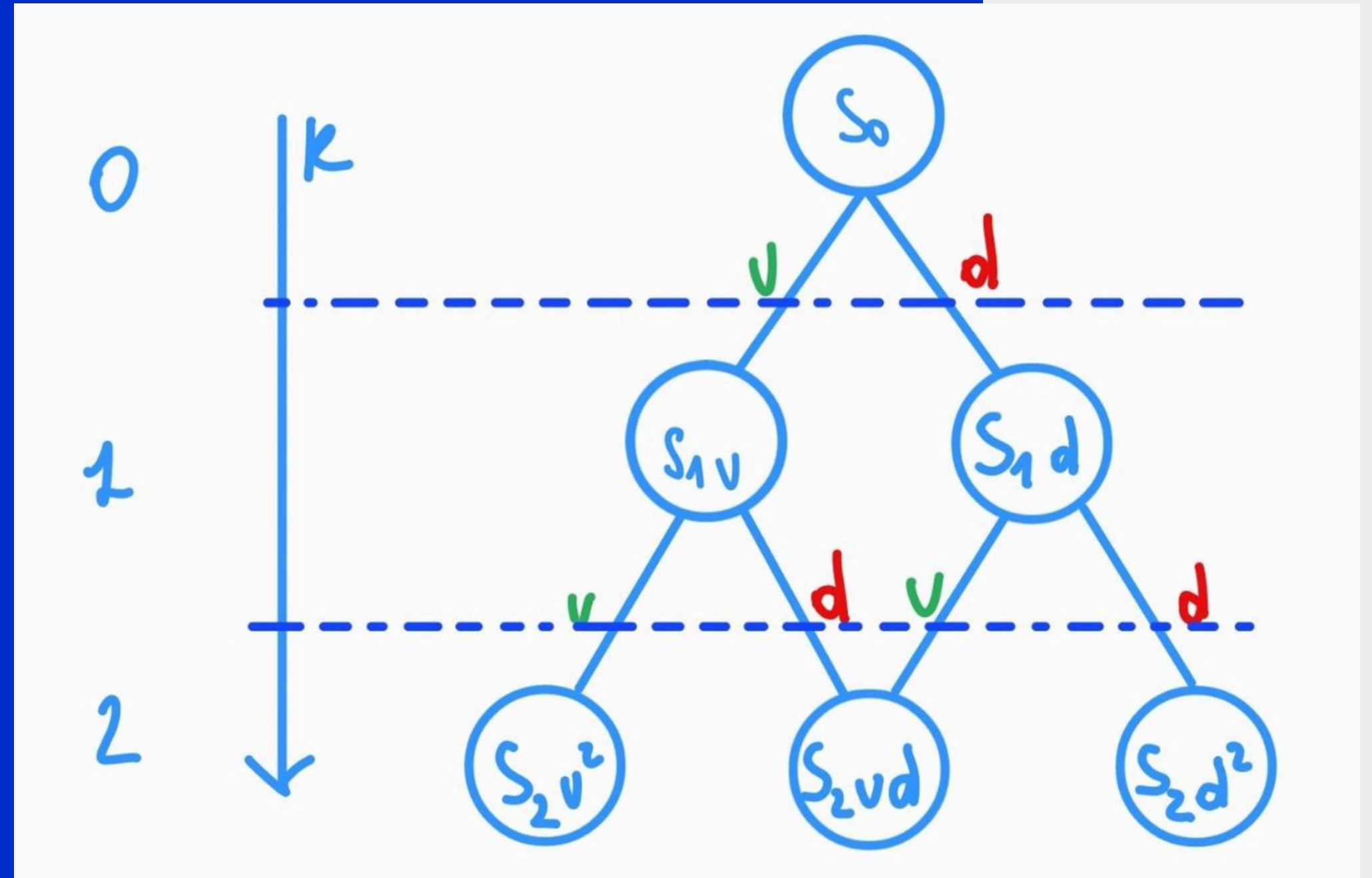
Alberi

- Sotto strutture fondamentali :
nodi e archi
- Ogni nodo ha massimo un
arco entrante
- Unico nodo privo di arco
entrante (Root)
- Nodi senza archi uscenti
(Leaves)



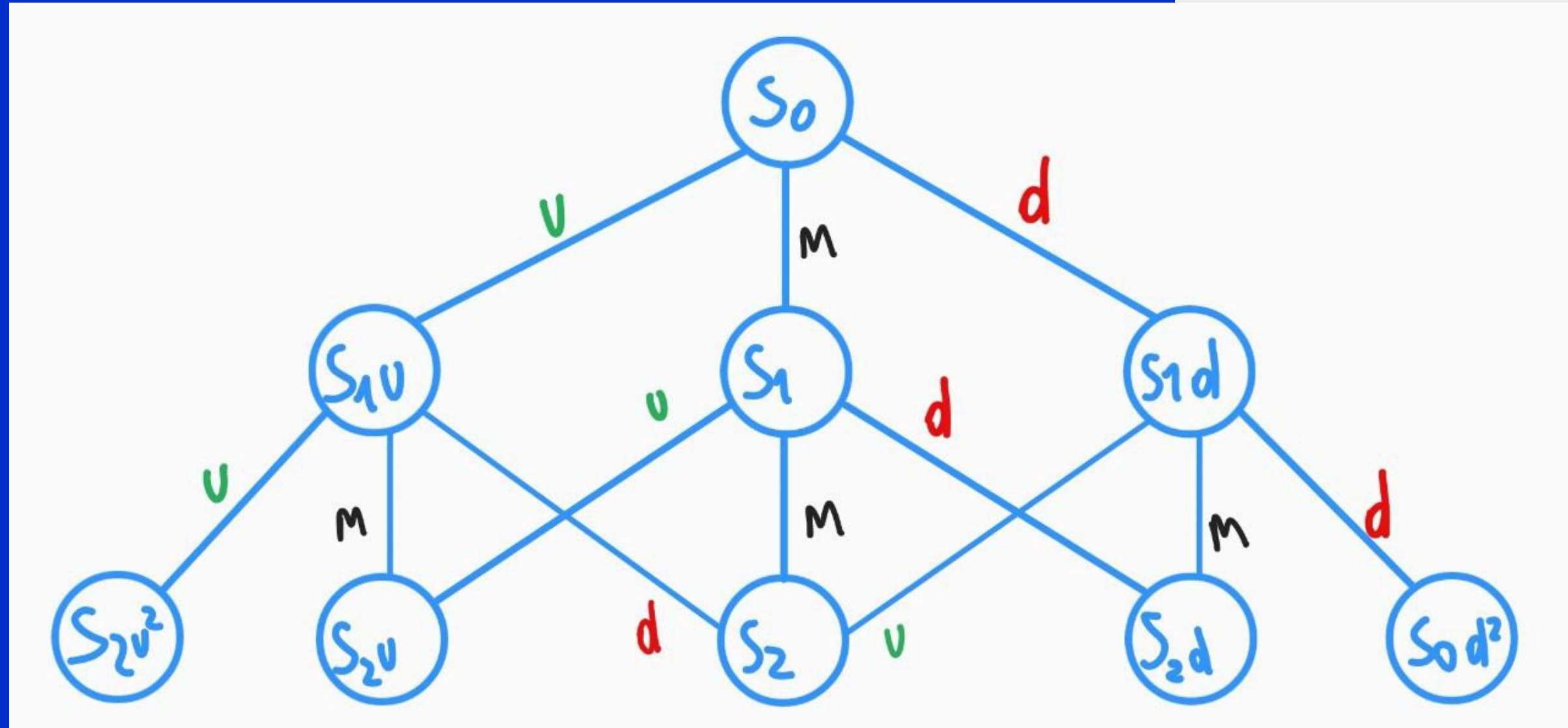
Alberi binomiali ricombinanti

- Struttura a livelli : al tempo k ci sono $k+1$ nodi
- Da 2^k cammini a $k+1$ nodi
- Ricombina perché l'ordine dei salti non conta ($ud = du$)
- Proprietà di Markov: $S_{k+1} \rightarrow S_k$
- Costo computazionale del tempo da $O(2^n)$ a $O(n^2)$



Alberi trinomiali ricombinanti

- Struttura a livelli : al tempo k ci sono $2k+1$ nodi
- Da 3^k cammini a $2k+1$ nodi
- Ricombina perché l'ordine dei salti non conta ($ud = du$)



- Costo computazionale del tempo da $O(3^n)$ a $O(n^2)$
- Proprietà di Markov : $S_{k+1} \rightarrow S_k$
- Transizioni : 3 esiti per passo con $p_u + p_m + p_d = 1$

Modelli utilizzati

CoxRossRubinstein: $u = e^{\sigma\sqrt{\Delta t}}, \quad d = e^{-\sigma\sqrt{\Delta t}}, \quad q = \frac{e^{r\Delta t} - d}{u - d}$

JarrowRudd: $u = e^{(r - \frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}}, \quad d = e^{(r - \frac{1}{2}\sigma^2)\Delta t - \sigma\sqrt{\Delta t}}, \quad q = \frac{1}{2}$

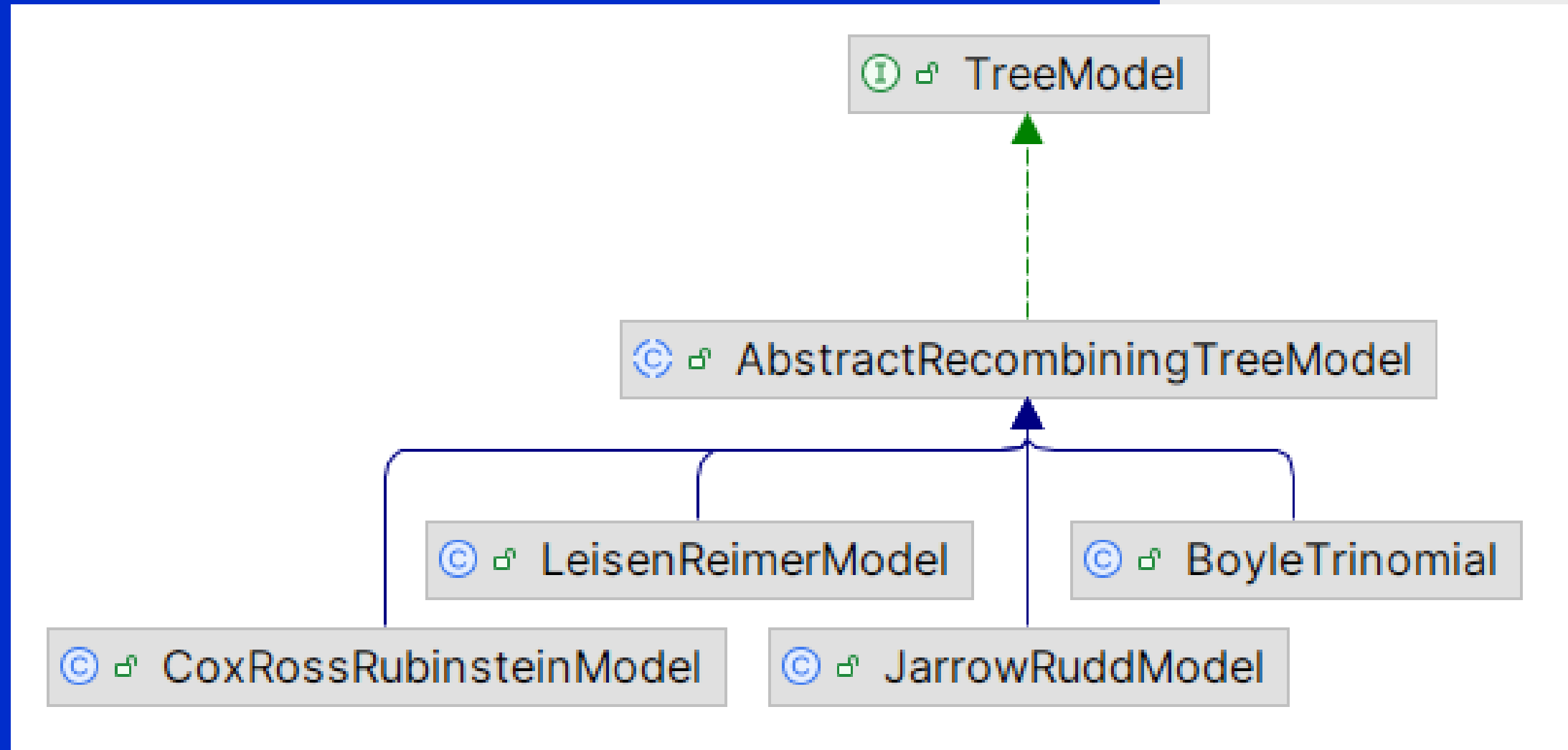
LeisenReimer: $u = e^{(r - \frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}}, \quad d = e^{(r - \frac{1}{2}\sigma^2)\Delta t - \sigma\sqrt{\Delta t}}$

Boyle: $u = e^{\sigma\sqrt{3}\Delta t}, \quad d = e^{-\sigma\sqrt{3}\Delta t}, \quad p_m = \frac{2}{3}, \quad p_u = \frac{1}{6} + \frac{(r - \frac{1}{2}\sigma^2)\sqrt{\Delta t}}{2\sigma\sqrt{3}},$

$$p_d = \frac{1}{6} - \frac{(r - \frac{1}{2}\sigma^2)\sqrt{\Delta t}}{2\sigma\sqrt{3}}$$

Gerarchia modelli

- Interfaccia Treemodel : ogni modello eredita dei metodi da implementare , ognuno dei quali differisce in base al modello utilizzato
- Tutti modelli fanno riferimento a una classe astratta che rappresenta un albero ricombinante
- Dalla classe astratta del modello ricombinante ereditano metodi helper da implementare per lo specifico modello



Programmazione dinamica

Condizioni necessarie :

- Sottoproblemi sovrapposti, risolvere stessi problemi più volte
- Struttura ottimale , composizione di soluzioni ottime di sottoproblemi

Ricorsione (Backward Induction) con tabulazione bottom-up

$$V_k = e^{-r\Delta t} \mathbb{E}^{\mathbb{Q}}[V_{k+1} \mid \mathcal{F}_k]$$

Optimal stopping per americane

$$V_k(s) = \max \left\{ \text{payoff}(s), e^{-r\Delta t} \mathbb{E}^{\mathbb{Q}}[V_{k+1} \mid S_k = s] \right\}$$

Sfruttare la struttura dati

Array per livelli + RandomVariable



Struttura ricombinante per costruzione



$(k+1 \parallel 2^{k+1})$ a livello k

$O(n^2)$ tempo e $O(n)$ memoria



Stato (k,i)

con $i \# \text{down}$

Array contiguo con accesso sequenziale

Niente pressione sul garbage collector

Evito problemi con puntatori (Pointer chasing)

Sfruttare la struttura dati

Struttura nodi e archi



Struttura non ricombinante, nodi duplicati raggiungibili da percorsi diversi



$O(n^2)$ tempo e memoria
ma costante più grande



Mappa per ricombinare

Per nodo : header oggetto (~16 B) + double S (~8 B) + 2
riferimenti figli (~16 B) ~40–56 B/nodo.

Sfruttare la struttura dati

$$M_{\text{nodi}} \approx 32 \cdot \frac{(N+1)(N+2)}{2} \approx 16 N^2 \text{ byte}$$

$$\sum_{k=0}^N 8(k+1) = 8 \cdot \frac{(N+1)(N+2)}{2} \approx 4 N^2 \text{ byte}$$

N	Numero nodi	Nodi + puntatori	Array per livelli
200	20 301	0,64 MB	0,16 MB
500	125751	3,81 MB	0,95 MB
1000	500500	15,3 MB	3,9 MB

Grazie per l'attenzione!